GINA CODY School of Engineering and Computer Science

Department of Computer Science and Software Engineering

Concordia University

SOEN 6441 -- Fall '26

Project Delivery# 1 (v1.1)

Due dates:
P1 (Team Information): 4 October 2026
P2 (Coding and demo): 9 November 2026

1. Overview

In this project, you will develop a web application using the Play Framework with Java that retrieves public Mastodon posts through the Mastodon REST API, processes the retrieved data, and presents the results through a web interface.

The project is divided into two deliveries. Delivery 1 focuses on building a conventional, non-reactive Play web application. In Delivery 2, you will extend your application to use Apache Pekko, actor-based processing, and WebSockets to provide near-real-time updates.

The two deliveries are intended to form a single evolving application. Therefore, you should design Delivery 1 with extensibility in mind.

For Delivery 1, do not implement WebSockets, actors, or reactive streaming. The application should use conventional HTTP request/response communication.

Please note that this assignment consists of both a group component and an individual component. Your group must submit a single application. Your marks will be based on the group work (50%) and your individual contribution (50%).

2. Scenario

2.1. Group Part 50%

Play Application: "MAWA" (Mastodon Analysis Web App)

Your application will allow a user to monitor and analyze posts associated with a Mastodon hashtag.

For example, the user could enter:

```
java
```

The application will retrieve posts associated with:

```
#java
```

from a Mastodon instance and present the results to the user.

The application should perform some processing and analysis of the retrieved posts, including sentiment analysis and statistical summaries (individual parts).

You are encouraged to think of the application as a small social-media monitoring and analysis tool.

3. Mastodon Account and Application

Each project team must create one dedicated Mastodon account for the project.

Do not use a personal Mastodon account for the application.

Each team must also register a Mastodon application and obtain a user access token with the required permissions.

The project should use the following minimum permission:

`read:statuses`

The access token must never be committed to your Git repository.

Your application should obtain configuration information from environment variables or another appropriate external configuration mechanism.

For example:

```
MASTODON_INSTANCE
MASTODON_ACCESS_TOKEN
```

Your source code must not contain an actual access token.

4. Functional Requirements

4.1 Hashtag Search

The application must provide a user interface allowing the user to enter a hashtag.

For example:

```
Hashtag: [ java              ]

            [ Search ]
```

The user should enter the hashtag without the # character.

The application must retrieve posts using the Mastodon REST API:

GET /api/v1/timelines/tag/{hashtag}

The application must not retrieve an unlimited number of posts.

The request should use the Mastodon limit parameter, with a maximum of 40 posts per request.

4.2 Display Retrieved Posts

The application must display the retrieved posts in a clear and usable format.

For each post, display at least:

- author;
- publication date/time;
- language
- post content;
- link to the original Mastodon post;
- visibility;

The application should present the post content as readable text rather than displaying the raw HTML returned by Mastodon.

You should not simply display the raw JSON response returned by the API.

The group part includes the setup of the Play framework for this web app, as well as a simple web interface that takes a string with a hashtag (without the # symbol).

You must use the official Mastodon API to search for items; no wrappers or libraries to handle the API are accepted.

Note: for this assignment, you do not need to stream updates to the user interface. In other words, the front-end page is static until a "refresh" (or new search) is triggered.

4.3. Individual Part 50%

In addition to the group part above, each team member has to work on *one* of the following features. Your team must decide who works on which part – two members cannot work on the same task!

If your team has fewer than five members, only pick one individual task per team member. Note that the individual part counts for 50% of the overall assignment marks.

a) **Post readability:** Display the item details and the overview's description readability. Calculate

the overview's readability using the Flesch-Kincaid Grade Level and the Flesch Reading Ease Score. The item details are shown on a new page when clicking on a result item.

b) Sentiment analysis: The app must perform sentiment analysis on the retrieved posts. Determine if their descriptions are overall happy ':-)', sad ':-(' or neutral ':-|'. This sentiment will be displayed on the main search page for each search query. Process the content field, finding happy and sad words, emojis, and emoticons and counting them (create your own "happy/sad" lists). For example, if a post contains more than 70% "happy" strings, return a happy sentiment :-), for more than 70% sad, return an unhappy emoticon :-(, otherwise a neutral one :-|.

You must use the Java Streams API to process the post's content.

c) Statistical Analysis: The application must provide aggregate information about the retrieved posts at the top of the search.

At minimum, display:

- total number of posts retrieved;
- number of positive posts;
- number of negative posts;
- number of neutral posts.

You must use Java Streams to perform at least part of the collection processing required for these statistics.

For example, the application could transform and aggregate the posts using operations such as:

- filter
- map
- groupingBy
- counting
- collect

d) Author Analysis: Add a link at the top of the search to identify the authors who contributed the largest number of posts in a new page.

e) Word stats: For a search query, display word-level statistics for the posts retrieved, counting all unique words in descending order (by frequency of the words). This statistics page must be hyperlinked from the search terms above the results. YYou must use the Java Streams API to process the post's content.

5. Coding guidelines.

Your submission must satisfy the following requirements (these apply to both the group and the individual work):

5.1 Pagination / Additional Results

The Mastodon API limits the number of statuses returned by a request.

Your application must provide a mechanism for retrieving additional results.

For example:

```
[ Previous ]    Page 1    [ Next ]
```

or:

```
[ Load More ]
```

The implementation should use the pagination mechanisms provided by the Mastodon API rather than downloading a large collection in a single request.

You should investigate the appropriate Mastodon parameters, such as max_id, for this purpose.

5.2 Error Handling

The application must handle failures gracefully.

At minimum, consider:

- invalid or empty hashtag input;
- Mastodon API unavailable;
- HTTP error responses;
- authentication failure;
- malformed or unexpected API data;
- no posts returned;
- sentiment-analysis failure.

The user should receive a meaningful message rather than a Java stack trace.

For example:

```
Unable to retrieve posts from Mastodon.
Please try again later.
```

The application should also use appropriate HTTP status codes when responding to requests.z

5.3 Play Framework.

1. Your application must be developed on the Play framework with Java.

2. It must be possible to build and run your app using the standard **sbt run** command.
3. Any required third-party libraries must be automatically resolved through **sbt**.
4. The standard build targets (sbt run/test/jacoco/javadoc/clean) must all work correctly.
5. Do **not** put business logic into the controller. Use dedicated model classes (MVC pattern) for your application functions.
6. Write *a single controller* for your app, integrating the individual parts defined below.
7. Do **not** query the API multiple times with the same hashtag. Develop an appropriate caching strategy for your app that stores the information to execute the different individual tasks.

You may set up a Continuous Integration server for your team (e.g., using Jenkins) or GitHub Actions, but that will not be evaluated.

5.4 Documentation.

The following guidelines concern your documentation:

1. Document all your classes and methods with *Javadoc*. If you use GenAI, do not forget to disclose it. For example, this class was generated with GenAI, using this prompt.
2. This includes private methods and unit tests!
3. Make sure all classes and methods include an **@author** tag (both for group and individual work).
4. Include the generated HTML Javadoc in your submission (make sure you generate it for the private methods as well).

5.5 Reactive Programming.

The following guidelines refer to using reactive programming techniques. These will become especially important for the second delivery (second part of the project), so you should do your best to get it right the first time.

1. Your controller actions must be *asynchronous* (non-blocking), using Java's CompletionStage<T> / CompletableFuture<T>.
2. Do not use .get()/.join(), thread.sleep, or similar to block for a future's result anywhere in your code (the only accepted exceptions are unit tests, see below).

Note: any blocking code in your app will lead to significant mark reduction!

5.6 Testing.

Testing your code is a very important part of the project. Make sure you write your unit tests at the same time as you implement your features. **Code that is not tested will count as not implemented!** – In other words, if you completely implement the whole project, but do not have a single test, your submission will receive 0 marks! Create JUnit tests for all your production classes. You must have tests for every method in your controller and every controller action, as well as any additional classes you wrote. Compute your test coverage with JaCoCo. You must reach 100% branch coverage of the classes and methods you wrote.

1. Within a test, you can use .get()/.join() to wait for a result.
2. **Never** call the live Mastodon API from a unit test; use a mock class for testing instead. For testing classes that call the Mastodon API, you must use either your own factory-generated mock class (you can use Dependency Injection (DI) with the Google Guice library) or use the *Mockito* framework.
3. Your unit tests must be properly designed by finding the equivalence classes and doing partition testing. A unit test that does not properly check the result values (e.g., only checking if a result is non-empty, rather than comparing the values) will count as *not written* for test coverage.

6. Submission.

For delivery 1, there are two deadlines: the first for submitting the information about your team and the individual task distribution; the second for delivering your web app. Note that for each submission, only one upload per team is required.

Submission part 1: Group Information. (1 point if submitted correctly and on time). You must submit the following team information on Moodle by the first due date:

GitHub: The URL of your GitHub repository containing the project.

Team setup: List all team members, indicating which of the individual parts above are assigned to each team member. An individual task can only be assigned to one team member, and the marks (50% of the total marks) for that task will go to the designated team member only.

Submission part 2: Code. You must submit your code electronically on Moodle by the due date (late submission will incur a penalty; see the course outline for details). Include a signed (by all team members) *Expectation of originality* form[1] with your submission.

Demo. You must also demo your code to your TA during labs (time slots for the demos will be given by your TA). **Note that all group members must be present for the demo and ready to answer questions about the project.**

Submission + Demo = 22 points. More details about the grading scheme can be found in the Rubric here.

7. What Delivery 1 Does NOT Require

The following are explicitly not required for Delivery 1:

- Apache Pekko actors;
- Pekko Streams;
- WebSockets;
- Mastodon streaming API;
- real-time updates;
- reactive programming;

[1] See https://www.concordia.ca/ginacody/students/academic-services/expectation-of-originality.html

- event-driven processing.

Do not implement these features as part of Delivery 1.

They will be introduced in Delivery 2.

8. Design for Delivery 2

Although you should not implement the reactive functionality yet, your Delivery 1 architecture must be designed so that it can be extended in Delivery 2.

In Delivery 2, the application will evolve toward:

```
Mastodon Streaming API
        ↓
Pekko
        ↓
Actor-based processing
        ↓
Play WebSocket
        ↓
Browser
```

Therefore, your Delivery 1 design should avoid making the Mastodon REST API an inseparable part of your entire application.

In particular, the application-level `Post` representation should remain independent of whether a post was obtained through REST or through the streaming API.

9. Git Repository

A Git repository containing the complete Play application.

The repository must include:

- source code;
- tests;
- Play configuration;
- templates/static resources;
- build configuration;

- README.

Do not include API credentials.

9.2. README

The README must explain:

- how to install/run the application;
- how to configure the Mastodon instance;
- how to configure the access token;
- how to use the application;
- how to run the tests;
- the main architectural decisions;
- any assumptions or limitations.

9.3. Architecture diagram

Provide an architecture diagram showing the main components and their relationships.

At minimum, show:

Browser

↓

Play Controller

↓

Service

↓

Mastodon Client

↓

Mastodon REST API

Also show the analysis components.

9.4. Short design document

Provide a short document explaining:

- your domain model;

- the responsibility of each major component;
- how Mastodon is isolated from the rest of the application;
- where Java Streams are used and why;
- how the design can evolve toward Delivery 2.

10. Demonstration

During the demonstration, your team should be prepared to show:

1. the application running;
2. a hashtag search;
3. retrieved Mastodon posts;
4. individual tasks;
5. retrieval of additional results;
6. handling of an API failure or invalid input;
7. relevant automated tests;
8. the application architecture;
9. where and how Java Streams are used.

You should be able to explain your design decisions and not simply demonstrate that the application works.

11. References.

1. Mastodon API. https://docs.joinmastodon.org/methods/timelines/
2. Flesch Kincaid Calculator. https://goodcalculators.com/flesch-kincaid-calculator/
3. Free sentiment analysis tool. https://text2data.com/Demo
4. Sentiment Analysis Word Lists Dataset https://www.kaggle.com/datasets/prajwalkanade/sentiment-analysis-word-lists-dataset?resource=download
5. SentiWordNet. https://github.com/aesuli/SentiWordNet